Hima — Support Bot

AI ticket-deflection feature — **how it works**, configuration, and operations

Innovfix Pvt. Ltd. · Hima App · v1 · 2026-07-17 · Backend: Laravel (demo_hima / hima-admin-panel) · App: Android (hima_app)

> **What it is, in one line.** A guided in-app chat that sits **in front of** the existing ticket system: it asks the user 3–4 questions, reads **their own live account data**, answers, and only creates a support ticket if the user says the answer did not help. The goal is to stop the repeat-contact tickets (the average user raised ~2 per month) at the source — without ever making a false statement about someone's money or account.

1. The user flow

Step	Screen	Notes
0 · Language	"Choose your language" (prompt in English, all 11 options)	Applies to **this conversation only** — does not change the profile language / matching.
1 · Category	5 categories, volume-ordered	Calls · Earnings/Withdrawal · Coins/Recharge · Account/Profile · Something else.
2 · Sub-category	The category's sub-options + "My issue isn't listed"	Chips are **server-driven** — the app renders labels and posts keys; it knows nothing about the taxonomy.
3 · One detail	A sub-category-specific question (optional, with Skip where safe)	e.g. "double charge" asks for a date/time; the answer genuinely needs it.
4 · Answer	Typing → the AI's answer	Only this step is slow. The bot has already read the user's data before replying.
5 · Did this solve it?	[Yes, solved] · [No, still need help]	**Yes** → close, **no ticket**. **No** → one more attempt, then a human.

> **Yes vs No — the whole point.** "Yes" closes the chat and **creates no ticket row** (a deflected issue must not appear in `tickets`, or deflection is unmeasurable). "No" buys **one** second, different answer; a second "No" creates a ticket for a human, pre-tagged and with the bot's diagnostics attached. There is no third attempt.

2. How it stays accurate (why it can be trusted with money)

- **Reads live data via fixed, read-only tools** — never free-form SQL. Eight tools: account summary, recent calls, call detail, recharge history, coin ledger, withdrawal history, earnings summary, block status.
- **The user is structural, not a parameter.** The logged-in user id is fixed on the server; it appears in **no** tool the model can call, so the model cannot express "look at another user's data" (closes IDOR by construction).
- **Grounding guard.** Every number in a reply (₹ amounts, coins, order ids, times) must be a value a tool actually returned. Any invented number → the answer is discarded and the issue is escalated.
- **Skip-AI on money.** For sensitive/redirect money categories the model is never called at all — a human takes it directly, so money cannot be hallucinated about.
- **Recharge crediting is never asserted as fact.** `transactions` has no gateway/order link and `add_coins` also covers admin grants, so a same-size credit near a payment is only an **indication**, not proof. The bot states **payment_status** (paid / not completed) as fact, reports any nearby credit as a labelled possibility with its confidence, and routes every paid recharge to a human to verify against the gateway dashboard — it never claims "you were credited" or "money taken, no coins".
- **Fail closed.** If a tool cannot read the user's data (DB/replica down), the bot escalates to a human rather than answering around the gap. On prod, reads use a replica; if the replica lags, it falls back to the primary so it never states stale data.
- **Never promises** a refund, credit or timeline; **never deletes** an account — a delete request is treated as a symptom and probed.

3. Configuration & kill-switches (ops)

Setting	Where	Effect
`support_bot_enabled`	`gateway_config`	**Master kill-switch.** When set to `0`, the app shows the old raise-ticket form and in-flight sessions escalate to a human instead of calling the model. Fails **closed** (unreadable = off).
`support_bot_model`	`gateway_config`	The model id (default Claude Sonnet). Deliberately separate from the ticket-answering model.
`known_issue_banner`	`gateway_config`	When set, shown first during a platform incident — deflects the duplicate flood before the menu.
`support_hours_start` / `_end`	`gateway_config`	Agent shift (default 4 PM–12 AM IST) for the out-of-hours "when we'll look" ETA.
`SUPPORT_BOT_READ_CONNECTION`	.env	Read replica for tool queries on prod (empty on demo). Lag-guarded.
`SUPPORT_BOT_SESSION_CEILING`	.env	Hard per-session deadline (25s) — under PHP's execution limit; the app waits longer so it always gets the definitive answer.
`SUPPORT_BOT_TOKEN_BUDGET`	.env	Hard per-session cost ceiling; exceeding it escalates rather than spending more.

> **Answers are ops-editable without a deploy.** The per-sub-category playbooks live in `ai_reference_faqs` (already per-language, already an admin UI). The Yes/No taps are a free labelled dataset — a sub-category below ~40% "Yes" is a broken playbook or a real product bug.

4. Endpoints (all `auth:api`, no user_id ever accepted from the client)

Endpoint	Purpose
`support_bot_start`	Opens/*resumes* a session; returns language chips + any known-issue banner. Checks for a pending escalated ticket up front.
`support_bot_reply`	Walks the tree, then answers. One request per session at a time (locked).
`support_bot_feedback`	"Did this solve it?" — the only thing that resolves or escalates. Runs the second attempt.
`support_bot_session`	Rebuilds a session reopened after the app was killed (transcript + current step).
`support_bot_attach`	Optional file on the ticket the bot raised (image/video/audio, ≤5MB). Serialised per ticket.

5. Rollout & what to watch

- **Demo-first, always.** Everything is verified on `demohima.himaapp.in` before prod. Prod goes on an explicit go, staged by sub-category allowlist and user %.
- **Fallback is safe.** With the kill-switch off, or on any start failure, the user gets the old form — the support channel is never removed behind the LLM.
- **Weekly metrics:** per-sub-category Yes-rate · grounding-guard trips (should be ~0) · p50/p95 latency · tokens/session · **48h re-contact rate** · and the north star: **tickets per user per month**.

> **Design guardrail — "calls not coming" (the #1 driver) is a real product bug the bot cannot fix.** The bot reads the data and escalates; it does **not** troubleshoot ("check your network"), which reads as being fobbed off by a robot. Its deflection target there is near-zero **by design** — track it as a product-bug signal, not a deflection KPI.

6. Technical inventory

Controllers, services & models

Component	Type	Responsibility
`SupportBotController`	Controller	The 5 endpoints; step machine; session lock; kill-switch guard; escalation + attach.
`SupportBotService`	Service	The model tool-loop; deadline budget; grounding guard; escalate control-flow.
`SupportToolRunner`	Service	The 8 read-only tools; user-scoped by constructor; replica-lag guard; recharge matcher.
`TicketCreationService`	Service	Turns a session into a human ticket; per-user advisory lock; canonical-phrase routing.
`SupportBotSafetyGate`	Service	Safety-only refusal (harassment/abuse/minors/self-harm/report).
`SupportHours · SupportBotStrings · SupportBotTaxonomy · Prompts\SupportBotPrompt`	Service	Shift/out-of-hours ETA · 11-language strings · categories & canonical phrases · the versioned system prompt.
`SupportBotSession`	Model	Per-conversation state (step, language, transcript, last_render, telemetry).
`Ticket · Users`	Model	Existing models; bot writes tickets with `source=bot`; reads Users for scope.

Tables (reads unless noted)

Table	Use
`support_bot_sessions` (RW)	Session state + telemetry. New table; migrations are `hasColumn`-guarded.
`tickets` (RW)	Human tickets on escalation (bot columns: source, sub_category, language, bot_session_id, screenshot).
`gateway_config`	Kill-switch, model id, known-issue banner, support hours. Runtime, cross-node.
`users · user_calls · transactions · coins · cashfree_payments · phonepe_payments · blocked_users`	Read-only tool sources (account, calls, ledger, packs, payments, block status).
`ai_reference_faqs`	Per-language answer playbooks (admin-editable, no deploy).

External providers · jobs

- **OpenRouter → Claude Sonnet** (via `gateway_config.support_bot_model`) — the only external call; synchronous, budgeted (25s ceiling, token cap), keyed by `openrouter_api_key`.
- **Cashfree / PhonePe** — **read-only**, via their payment tables; the bot never calls the gateways.
- **Jobs/cron:** the bot adds **none** — every request is synchronous. Escalated tickets flow through the existing ticket pipeline. No queue worker to run or monitor.

Failure modes & handling (all fail toward a human, never a wrong answer)

Failure	Handling
Model timeout / API error / round limit	Escalate to a human ticket (deadline budget → `escalate`).
Tool can't read data (DB/replica down)	Fail closed — escalate (`tool_error`), never answer around the gap.
Replica lagging	Read from the primary for that session.
Ungrounded number in the reply	Discard the answer, escalate (grounding guard).
Recharge crediting unprovable	Report payment_status only; route to a human to verify (no credited/no-coins claim).
Kill-switch off (start or mid-session)	Old raise-ticket form / escalate the in-flight session; no model call.
Duplicate ticket race / concurrent attach	Per-user & per-ticket MySQL advisory locks (cross-node).
Node-local attachment storage	**Known operational risk** — an upload may be unavailable from another node until ticket storage is moved to a shared disk (platform task).

Rollback

- **Instant:** set `gateway_config.support_bot_enabled=0` — users get the old form, in-flight sessions escalate. No deploy, cross-node.

- **Staged:** sub-category allowlist + user-% for a gradual turn-up/down.
- **Schema:** all new columns are additive and `hasColumn`-guarded; reverting the branch leaves the tables harmless. No destructive migration.

7. QA test cases (formal — run before release)

How to run. Each case has a stable ID (`SB-###`), priority (P1 blocks release), module reference, preconditions and test data, numbered steps, and the expected result. Fill the **Execution** row: mark Pass/Fail, attach Evidence (screen recording / DB row), log a Defect ID on failure, and record the Retest result. **Build/role/data:** run on the Play/Internal build, as a **male caller** and a **female creator**, on WiFi **and** mobile data, unless a case says otherwise.

SB-001 P1 Guided flow, end to end

Module: Support Bot / Flow · Ref §1

Precondition	Bot enabled (`support_bot_enabled=1`); test account has no open escalated ticket.
Test data	Any account; pick category "Calls".
Steps	1. Open Help → Support. 2. Observe the language prompt; select one. 3. Select a category, then a sub-category. 4. Answer the follow-up (or Skip if offered). 5. Wait for the answer, then observe the "Did this solve it?" controls.
Expected	Steps appear in order language → category → sub-category → optional detail → answer → Yes/No. "My issue isn't listed" is present at the sub-category step. No crash; each step responds <2s except the answer.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-002 P1 Language & script consistency

Module: Support Bot / Language · Ref §1.0

Test data	Select English at step 0; then type the free-text detail in Tanglish ("calls varala").
Steps	1. Select a language and complete the flow to an answer. 2. Read every bot message for language consistency. 3. At the free-text step type romanised (Tanglish) text.
Expected	The whole conversation stays in the chosen language (no mid-chat switch). The answer mirrors the user's script (Tanglish in → Tanglish out), not native Tamil.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-003 P1 Deflection creates no ticket

Module: Support Bot / Deflection · Ref §1.5

Precondition	DB/admin access to check the `tickets` table.
Steps	1. Complete a flow to an answer. 2. Tap "Yes, solved". 3. Query `tickets` for this user/timestamp.
Expected	Conversation closes; optional rating shows; **no new row** in `tickets` .

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-004 P1 Escalation: second attempt then human

Module: Support Bot / Escalation · Ref §1.5

Steps	1. Reach an answer; tap "No, still need help". 2. Observe the second attempt (a different answer). 3. Tap "No" again. 4. Open the ticket in admin.
Expected	Exactly one second attempt (different content). A ticket is created for a human (`source=bot` , `requires_manual_support=1`) with the bot's diagnostics attached. No third AI attempt.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-005 P1 **Delete-account is probed, never actioned** Module: Support Bot / Safety · Ref §2

Steps	1. Choose Account → a delete-account intent (or type "delete my account"). 2. Follow the conversation.
Expected	The bot asks what the underlying problem is and tries to help; it never offers or performs deletion. No account is deleted or disabled.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-006 P1 **Resume mid-flow after app kill** Module: Support Bot / Resume · Ref §4

Steps	1. Advance to the sub-category / detail step. 2. Force-kill the app (swipe from recents). 3. Reopen Help → Support.
Expected	The prior conversation is rebuilt and lands on the same step; the user does not re-answer earlier questions.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-007 P2 **Resume after answer / when finished** Module: Support Bot / Resume · Ref §4

Steps	1. Receive an answer (Yes/No shown); kill and reopen. 2. Resolve or escalate the session; kill and reopen again.
Expected	First reopen shows THAT answer with Yes/No (never the rejected first answer). After resolve/escalate, reopen is read-only (transcript only, no live input).

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-008 P2 **Attachment — validation & lifecycle** Module: Support Bot / Attach · Ref §4

Precondition	A ticket has just been raised (SB-004), so the attach option is offered.
Test data	A <5MB image; a >5MB video; a .txt file; a large screen-recording.
Steps	1. Attach the valid image → confirm it appears on the ticket. 2. Attach the >5MB file → observe rejection. 3. Attach the .txt → observe type rejection. 4. Start a large upload, then rotate the screen; and separately, start one and background/kill the app. 5. Try to pick a second file while one is uploading.
Expected	Valid file uploads (visible on the ticket in admin). Too-large/wrong-type are rejected with a clear message and are NOT shown as accepted. Rotation mid-upload does not break the upload or crash. Killing mid-upload leaves no orphaned cache file. A second pick while uploading is blocked with a "wait" message.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-009 P2 **Slow-network patience** Module: Support Bot / Timeout · Ref §1.4

Test data	Throttle the network (e.g. slow 3G profile).
Steps	1. Reach the answer step on a slow connection. 2. Repeat for the second attempt (after "No").
Expected	The app waits (up to ~30s) for the definitive answer or escalation on BOTH the first and second attempts; it does not fail early or dump the user to the old form.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-010 P1 **Kill-switch (start & mid-session)** Module: Support Bot / Ops · Ref §3

Precondition	Ability to set `gateway_config.support_bot_enabled`.
Steps	1. Set the flag to 0; open Help → Support. 2. Set it back to 1; start a session and reach the sub-category step. 3. Set it to 0 mid-session; send the next step.
Expected	With the flag 0, the old raise-ticket form is shown (no bot). Flipping to 0 mid-session makes the next step escalate to a human (no AI call), with no hang or crash.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-011 P2 **Pending ticket & duplicate guard** Module: Support Bot / Tickets · Ref §1

Precondition	The test account already has one open escalated ticket.
Steps	1. Open Help → Support. 2. Separately, from two devices, drive one session each to escalation at the same time.
Expected	The user is told about the open ticket at the START (not after four questions). No duplicate ticket is created, even from two concurrent devices.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-012 P1 **MONEY — recharge is verified, never invented** Module: Support Bot / Money · Ref §2 · MUST-PASS

Precondition	DB access; a creator and a male account. Verify every figure against the DB, never the reply alone.
Test data	A real ₹1–₹49 recharge; also one completed >30 min after starting the order.
Steps	1. Ask a recharge/coins question after a real recharge. 2. Compare the bot/ticket diagnostics to the `cashfree_payments` / `phonepe_payments` and `transactions` rows. 3. Repeat for a delayed (>30 min) completion and for two close same-size purchases.
Expected	Recharge routes to a human. Diagnostics state **payment_status** as fact only; crediting is shown as INDICATIVE (never a definitive "you were credited" or "money taken, no coins"). A delayed credit is still matched; two close same-size purchases with one credit are marked for human review, not falsely credited. Every ₹/coin/order-id/time in any reply came from the data (no invented refund/timeline).

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-013 P1 **MONEY — earnings connected count** Module: Support Bot / Money · Ref §2 · MUST-PASS

Precondition	A creator account with some calls that never connected (`started_time='00:00:00'`).
Steps	1. Ask an earnings question. 2. Compare the "connected calls" figure to the Recent Calls tool and the DB.
Expected	The connected count excludes `00:00:00` (never-connected) calls and matches Recent Calls exactly.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-014 P1 **SECURITY — own data only (IDOR)** Module: Support Bot / Security · Ref §4 · MUST-PASS

Precondition	Two accounts A and B; a tool to send raw API requests (Postman).
Steps	1. As A, drive a bot session; confirm it only ever reflects A's data. 2. With A's token, call the ticket routes passing B's user_id / mobile number.
Expected	The bot only ever shows A's own data. Ticket create/list use the token identity — passing B's user_id/mobile returns A's own tickets or 401, never B's data; unregistered vs registered numbers return identical responses (no enumeration).

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____

SB-015 P3 **Zoho launcher hygiene** Module: Support Bot / UI · Ref §5

Steps	1. As a female account (launcher enabled from home), open Support and leave. 2. As a male account, open Support and leave.
Expected	Female: the floating Zoho launcher is hidden on the bot screen and restored on exit. Male: the launcher never appears.

Execution: □ Pass □ Fail **Evidence:** _____ **Defect#:** _____ **Retest:** _____